

2025-2027 Cloud specialist

UF: Containers-Microservices-Servlerless

Docente: Denis Maggiorotto

Communication patterns

Questi i pattern di comunicazione più utilizzati:

- 1-to-1 sync

- 1-to-1 async

 - Push

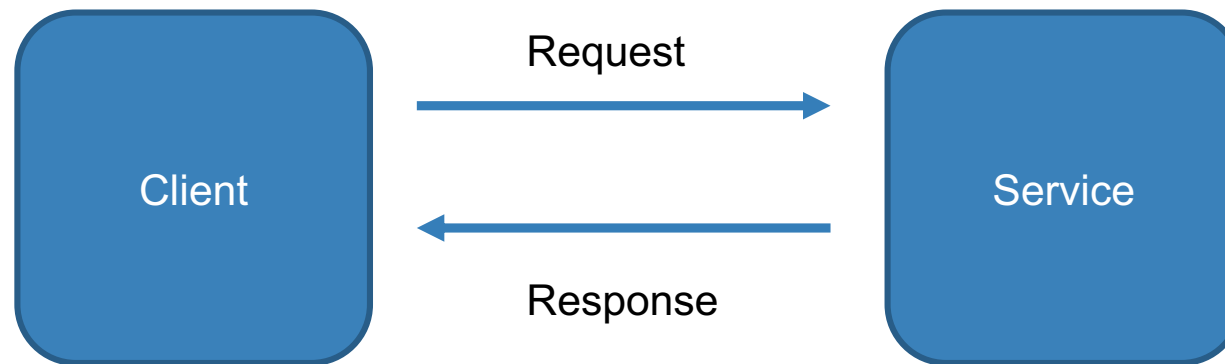
 - Pull

- 1-to-1 one way

- Pub-Sub/Event driven

Communication patterns

1-to-1 sync



Communication patterns

1-to-1 sync:



Pro:

Risposta immediata

Ottima gestione degli errori

Facile da implementare

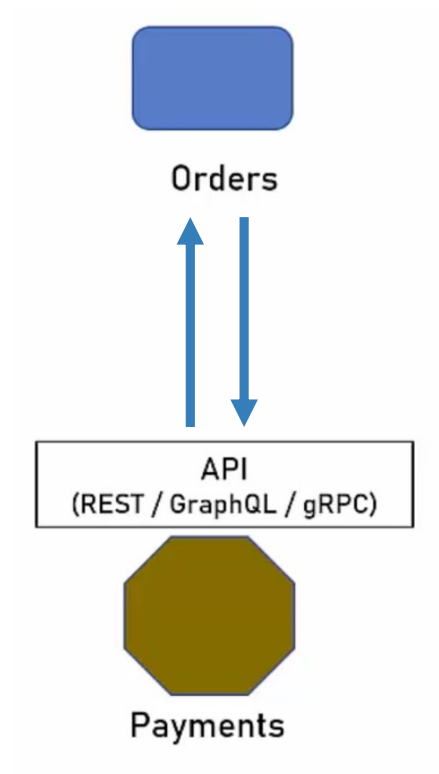


Contro:

Performance

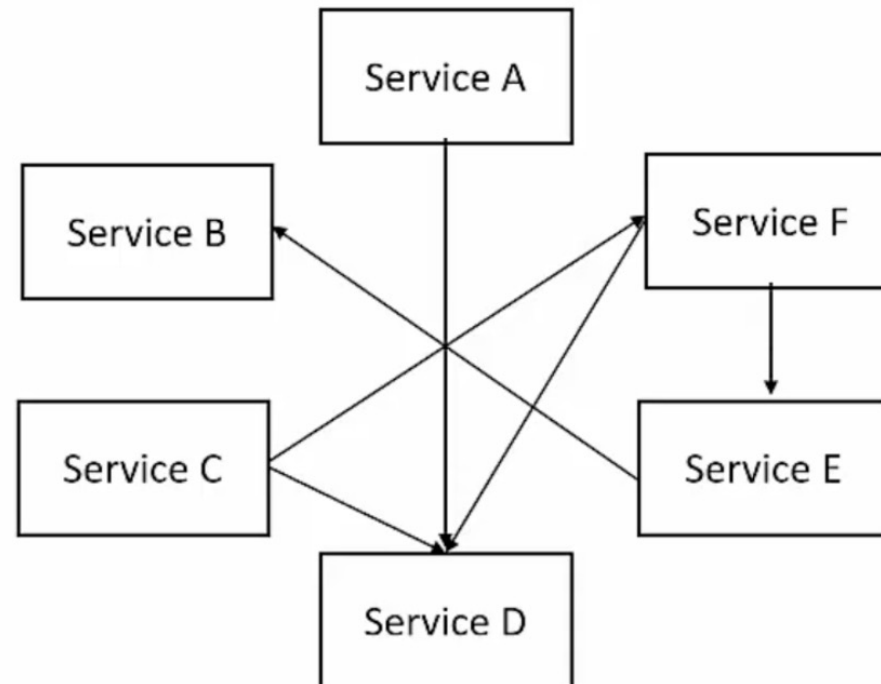
Communication patterns

1-to-1 sync:

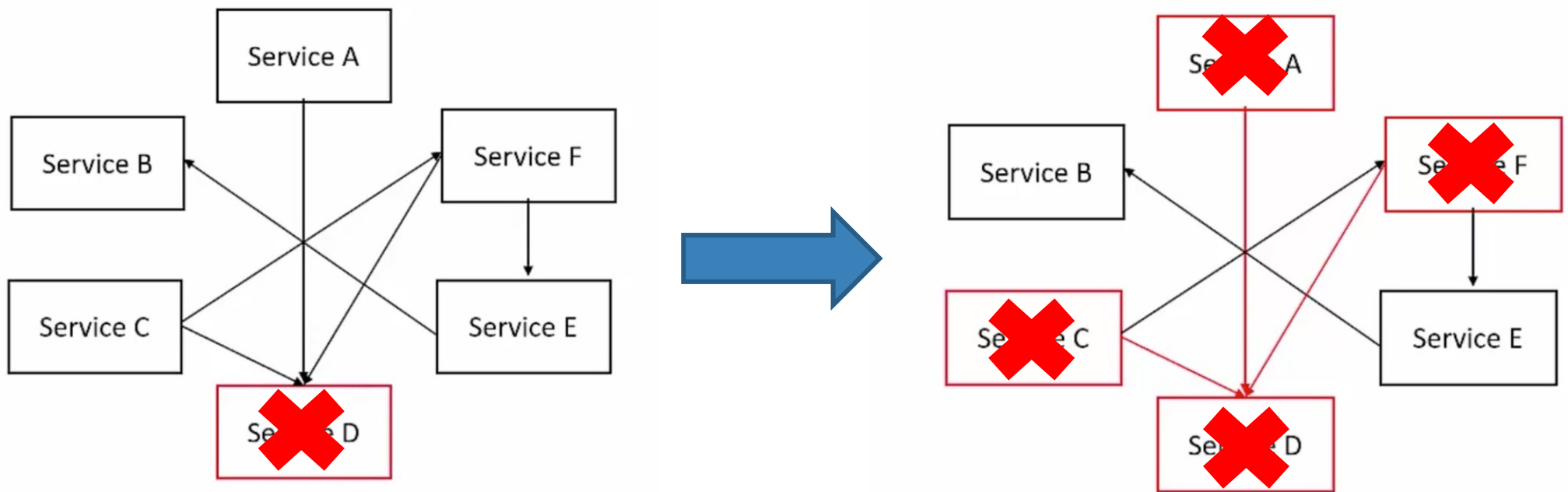


Communication patterns

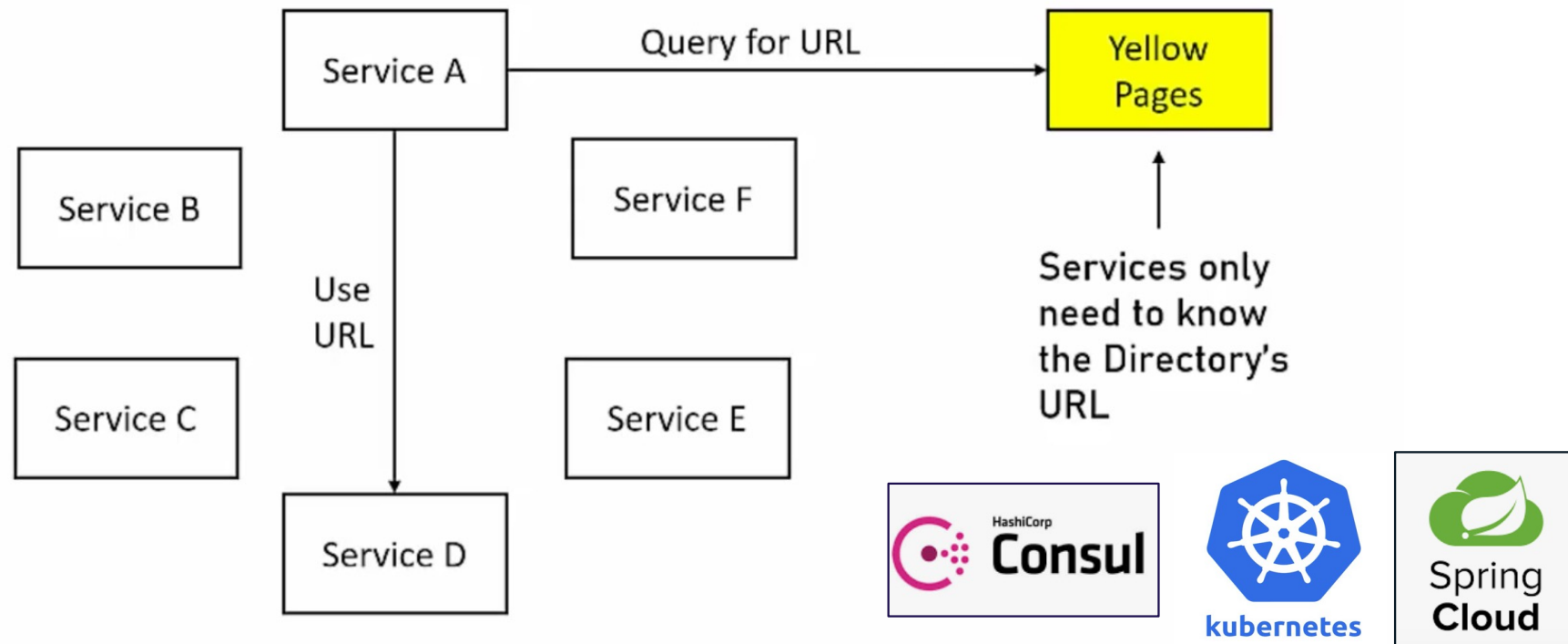
❌ Meglio evitare
connessioni dirette di
tipo **1-to-1 sync**



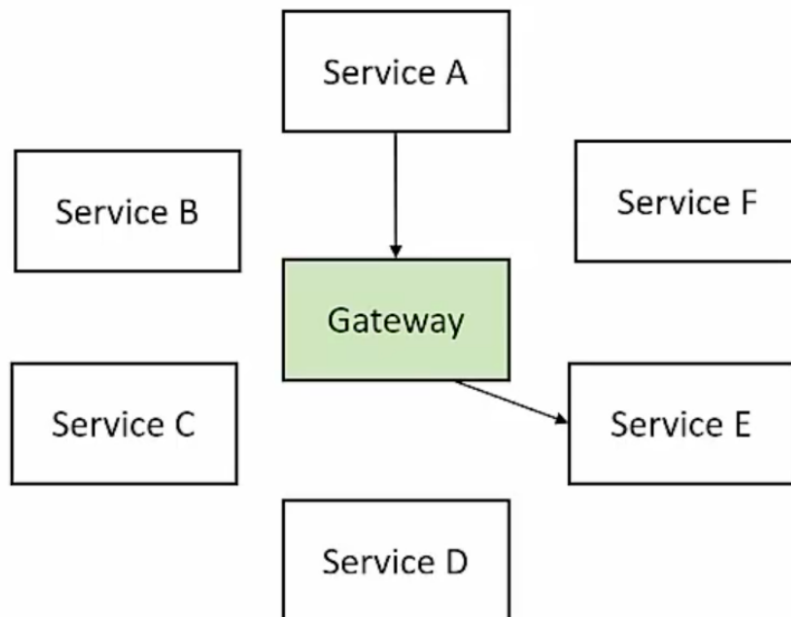
Communication patterns



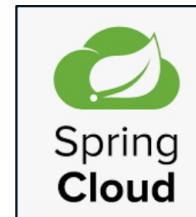
Communication patterns



Communication patterns

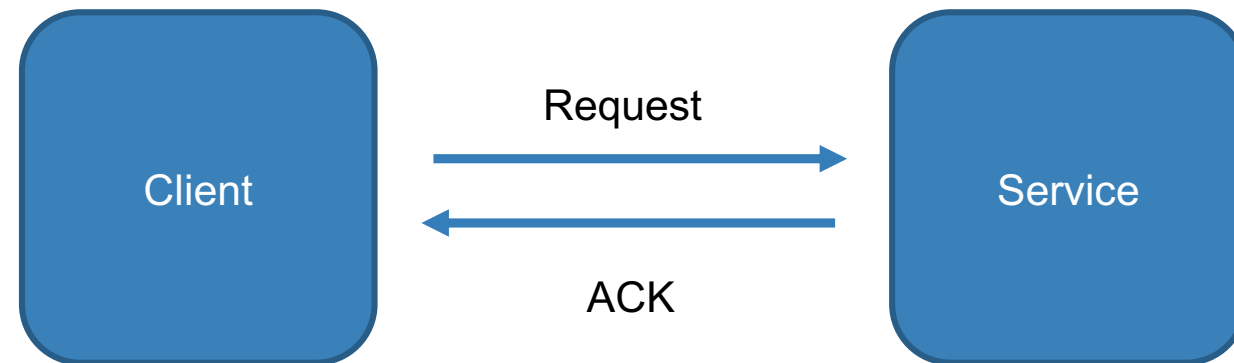


Services only
need to know
the Gateway's
URL

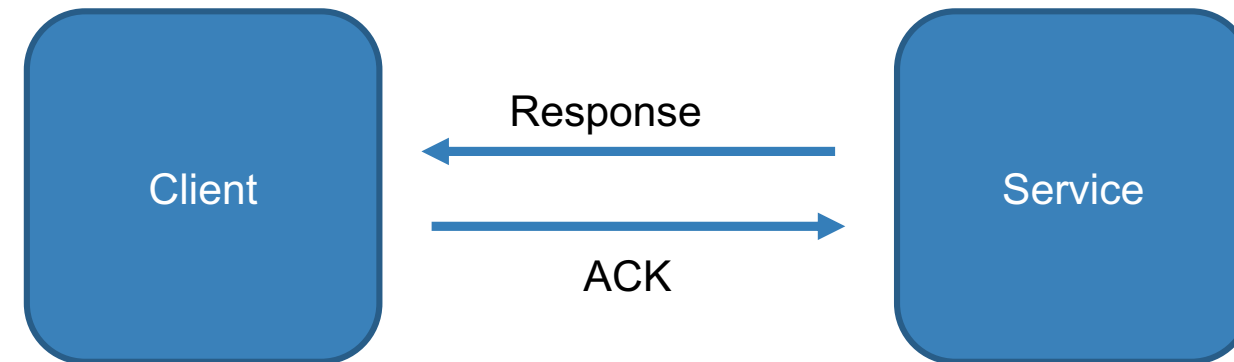


Communication patterns

1-to-1 async (push)

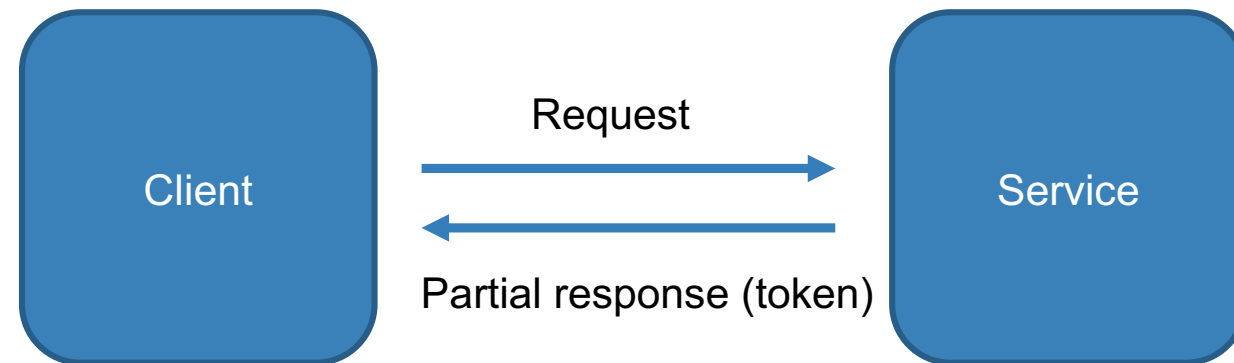


Più avanti nel tempo

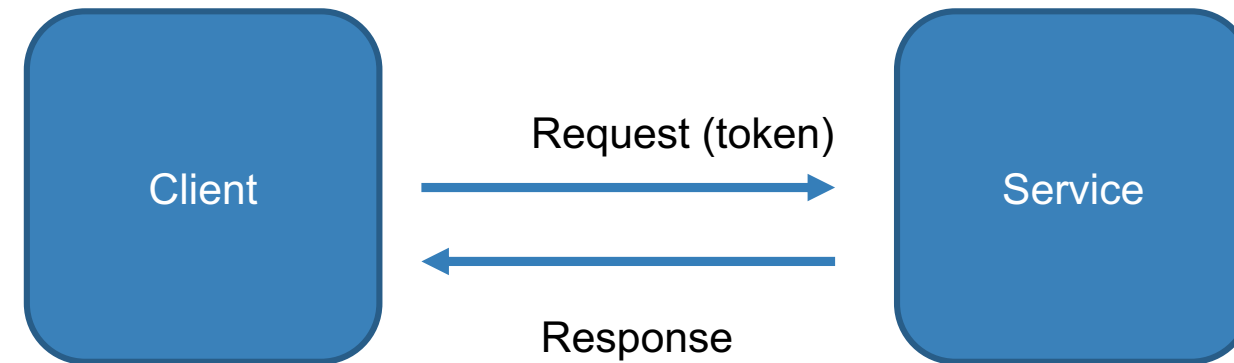


Communication patterns

1-to-1 async (pull)



Più avanti nel tempo



Communication patterns

1-to-1 async:



Pro:

Performance

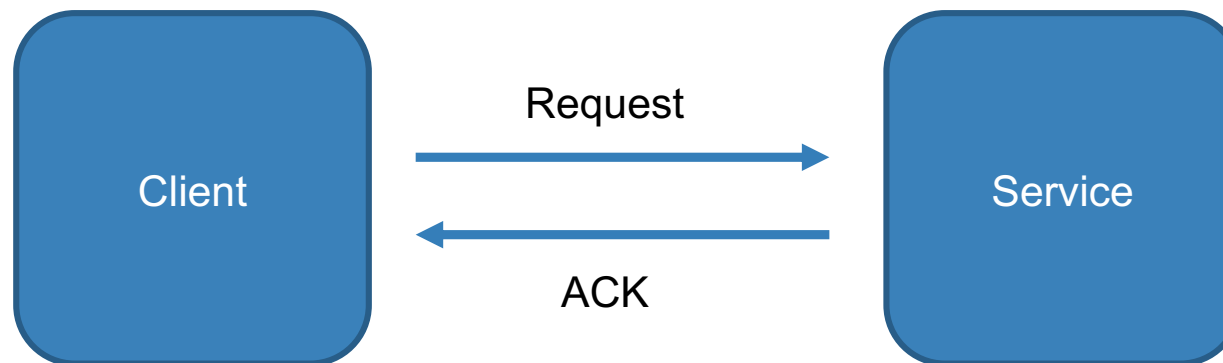


Contro:

Più complessa da implementare

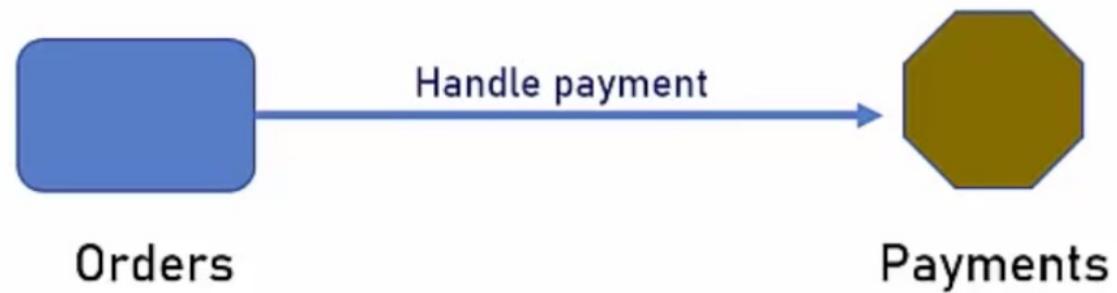
Communication patterns

1-to-1 one way



Communication patterns

1-to-1 one way



Communication patterns

1-to-1 one way:



Pro:

Ottima performance

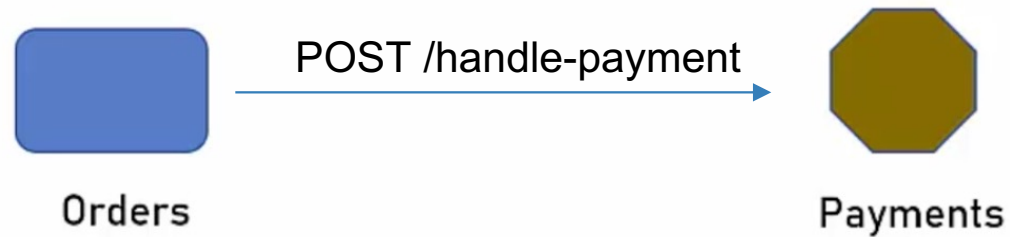


Contro:

Gestione degli errori
difficoltosa

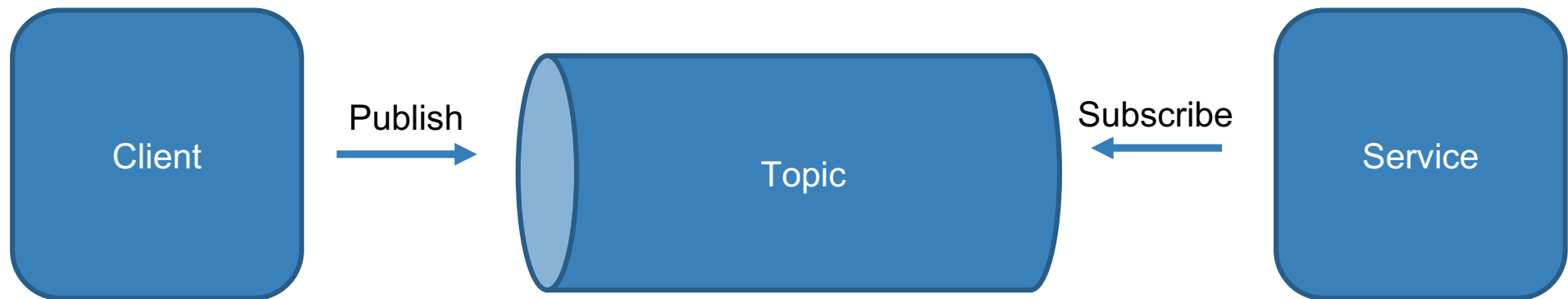
Communication patterns

1-to-1 one way using HTTP



Communication patterns

Pub-sub / Event driven



Communication patterns

Pub-sub / Event driven:

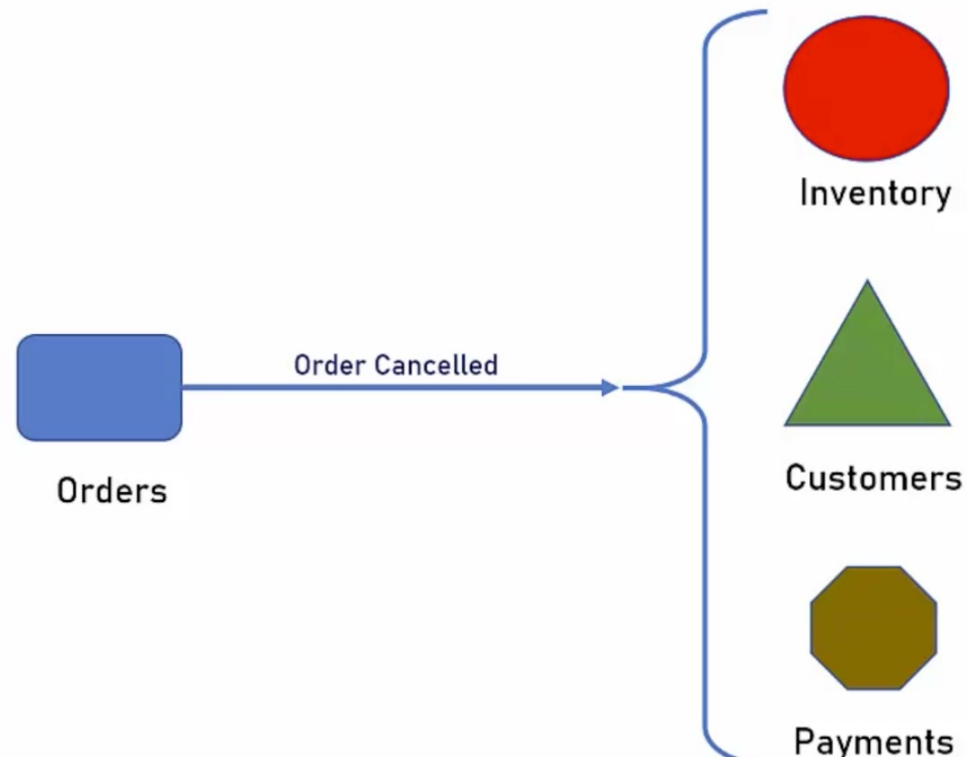
Un servizio vuole notificare altri servizi che un evento è avvenuto

Il servizio che invia il messaggio non ha idea di chi siano ne quanti siano i riceventi

Il servizio che invia non aspetta risposta (one way)

Communication patterns

Pub-sub / Event driven:



Communication patterns

Pub-sub / Event driven:



Pro:

Ottime performance

Ottima per le comunicazioni da
1 a molti



Contro:

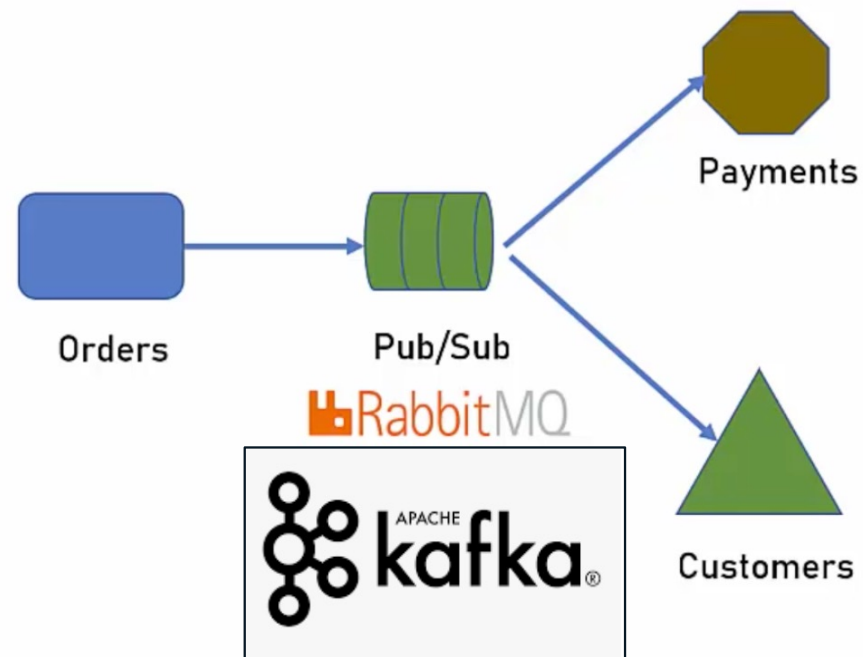
Implementazione più complessa

Spesso necessita di message
brokers a supporto

Gestione degli errori difficoltosa

Può causare un sovraccarico
all'infrastruttura

Communication patterns



Communication patterns

Communication patterns:

Influiscono:

- Le performance

- La gestione degli errori

- Il flusso di esecuzione

E' praticamente impossibile sostituire un pattern ad applicazione realizzata

Technology stack

La conformazione decentralizzata dell'applicazione rende possibile la scelta di tecnologie differenti per ogni microservizio

Spesso il focus è sulle tecnologie di backend e database

Non ci sono scelte giuste o sbagliate a priori

Occorre scegliere sulla base di evidenze tangibili

Backend technology stacks

Tipologia di applicazione

Type system

Cross platform

Community

Performance

Curva d'apprendimento

Backend technology stacks

	App Types	Type System	Cross Platform	Community	Performance	Learning Curve
.NET	All	Static	No	Large	OK	Long
.NET Core	Web Apps, Web API, Console, Service	Static	Yes	Medium and growing rapidly	Great	Long
Java	All	Static	Yes	Huge	OK	Long
node.js	Web Apps, Web API	Dynamic	Yes	Large	Great	Medium
PHP	Web Apps, Web API	Dynamic	Yes	Large	OK -	Medium
Python	All	Dynamic	Yes	Huge	OK -	Short

Database technology stacks

Relational DB

NoSQL DB

Cache

Object store

Relational DB

Immagazzina i dati in tabelle

Le tabelle hanno delle colonne

Le tabelle hanno relazioni con altre tabelle (PK,FK)

Modello transazionale ACID

Relational DB

Column Name	Type	Nullable?
OrderId	Numeric	No
OrderDate	DateTime	No
CustomerId	Numeric	No
DeliveryAddress	String	No

Column Name	Type	Nullable?
OrderItemId	Numeric	No
OrderId	Numeric	No
ItemName	String	No
Quantity	Numeric	No

Relational DB



Relational DB



E' la scelta giusta quando:

I dati sono fortemente strutturati

Serve un sistema di query molto potente

NoSQL DB

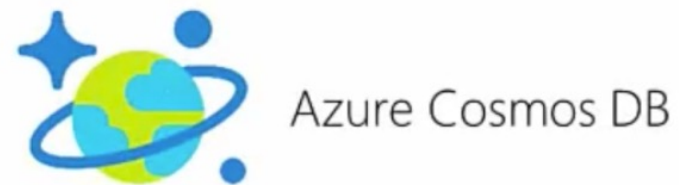
Enfasi su scaling e performance e meno su struttura dei dati e capacità di interrogazione

Modello transazionale BASIC

I dati sono semi-strutturati (schema-less)

I dati sono spesso immagazzinati nel formato JSON

NoSQL DB



NoSQL DB



E' la scelta giusta quando:

Quando i dati sono semi-strutturati

Servono scaling e performance

Non serve fare query complesse (join)

Cache

Immagazzina i dati in memoria per un accesso più veloce e performante

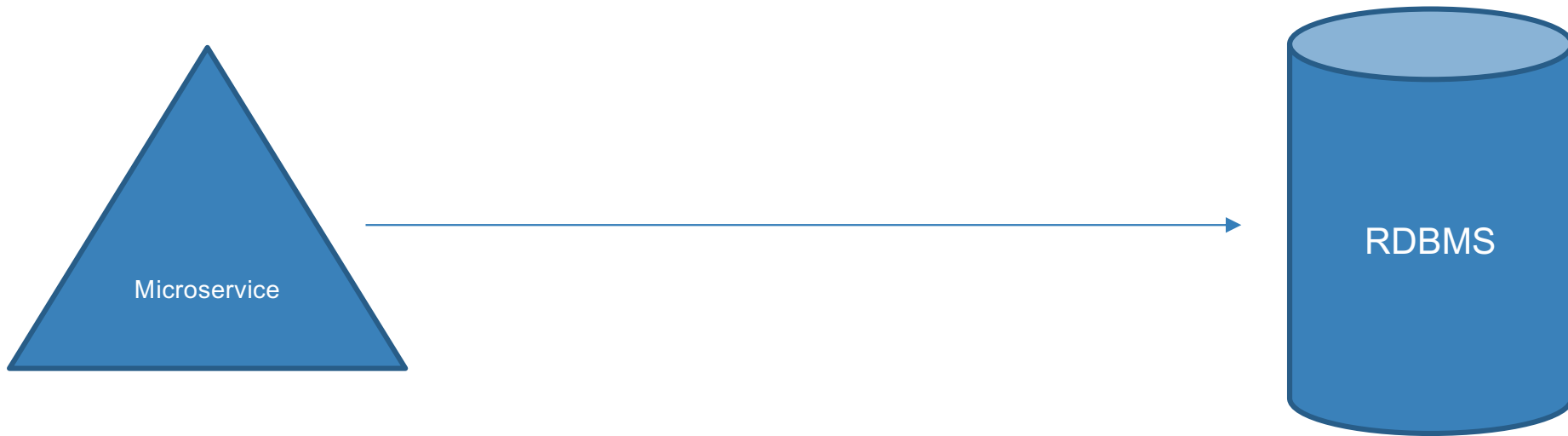
Spesso usato in congiunzione con altri DB (RDBMS)

Usato per memorizzare dati richiesti con grande frequenza e che cambiano di rado

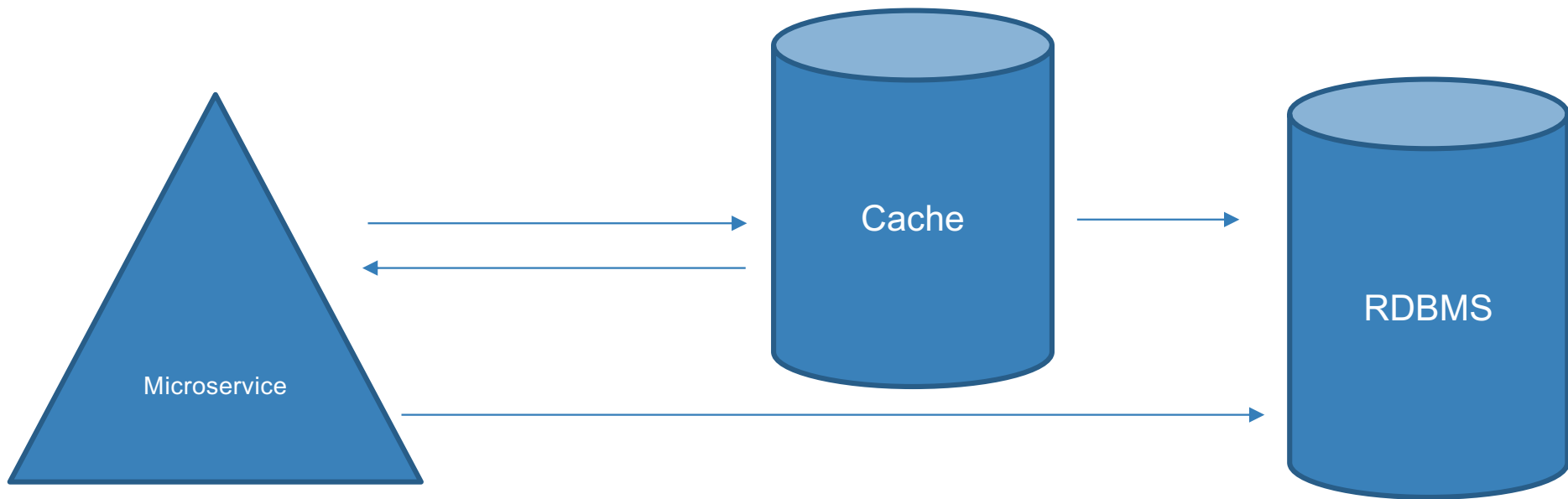
Distribuiscono i dati tra i nodi del cluster (distributed cache)

Immagazzina solo dati serializzabili

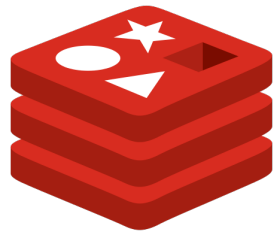
Cache



Cache



Cache



redis



hazelcast

Object store

Utilizzati per immagazzinare dati destrutturati

Documenti

Foto

Files

Object store

Microsoft Azure
Blob Storage



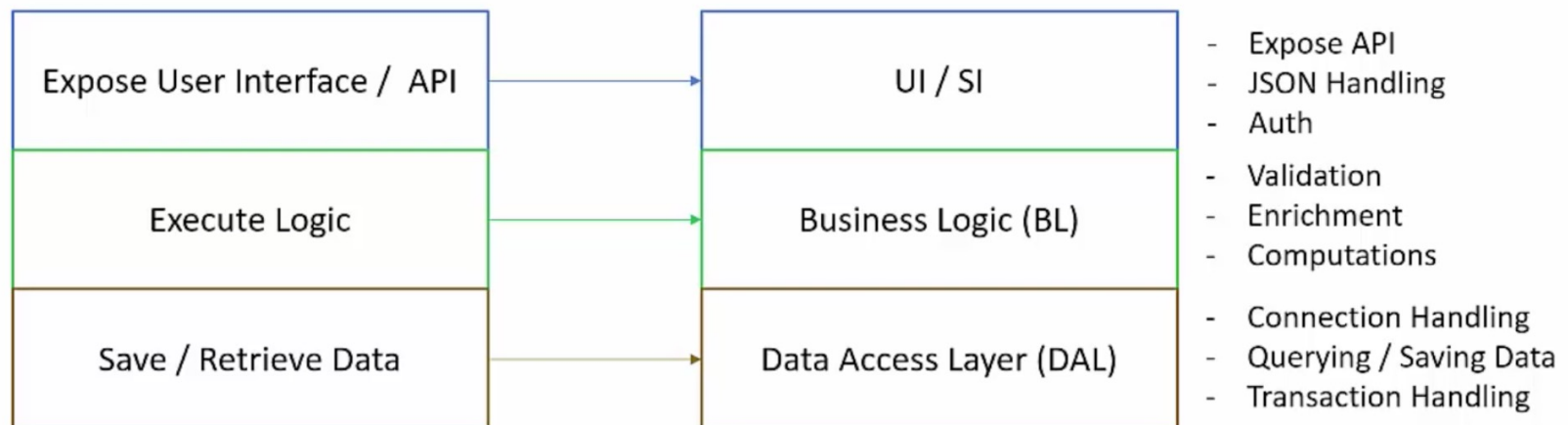
amazon
S3

MINIO

Architecture design

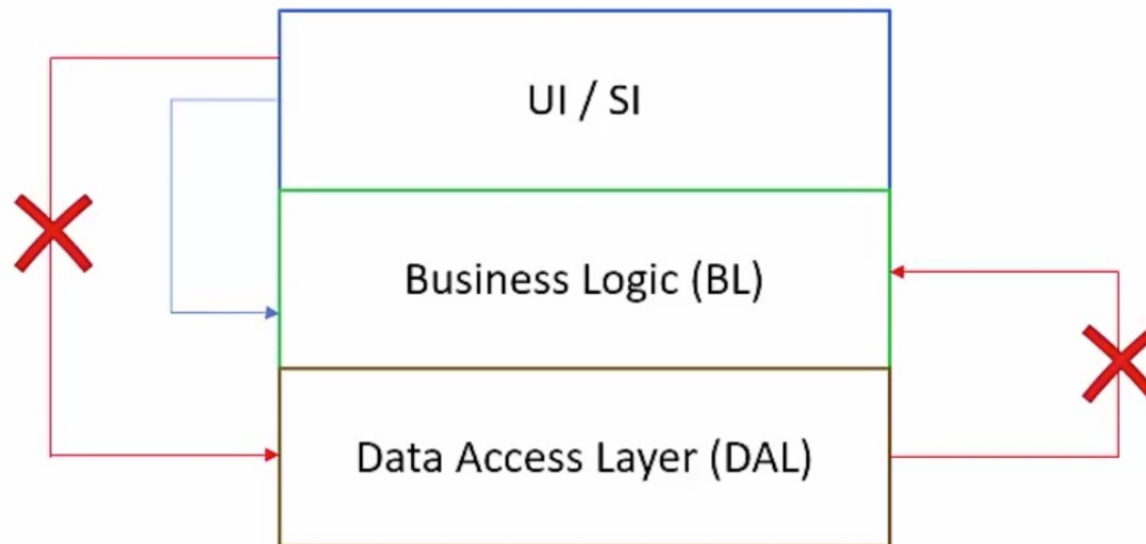
L'architettura dei servizi non differisce dall'architettura del software tradizionale (ogni servizio è un software)

E' basata sul layers pattern



Layers pattern

Ogni layer è focalizzato su di un ruolo specifico
I layer rendono il codice modulare



12 factors

<https://12factor.net>



THE TWELVE-FACTOR APP

INTRODUCTION

In the modern era, software is commonly delivered as a service: called *web apps*, or *software-as-a-service*. The twelve-factor app is a methodology for building software-as-a-service apps that:

- Use **declarative** formats for setup automation, to minimize time and cost for new developers joining the project;
- Have a **clean contract** with the underlying operating system, offering **maximum portability** between execution environments;
- Are suitable for **deployment** on modern **cloud platforms**, obviating the need for servers and systems administration;
- **Minimize divergence** between development and production, enabling **continuous deployment** for maximum agility;
- And can **scale up** without significant changes to tooling, architecture, or development practices.

The twelve-factor methodology can be applied to apps written in any programming language, and which use any combination of backing services (database, queue, memory cache, etc).

BACKGROUND

The contributors to this document have been directly involved in the development and deployment of hundreds of apps, and indirectly witnessed the development, operation, and scaling of hundreds of thousands of apps via our work on the Heroku platform.

This document synthesizes all of our experience and observations on a wide variety of software-as-a-service apps in the wild. It is a triangulation on ideal practices for app development, paying particular attention to the dynamics of the organic growth of an app over time, the dynamics of collaboration between developers working on the app's codebase, and avoiding the cost of software erosion.